Software Design Patterns & Frameworks

Lecture 1 - Introduction to Design Patterns Software Design Patterns & Frameworks

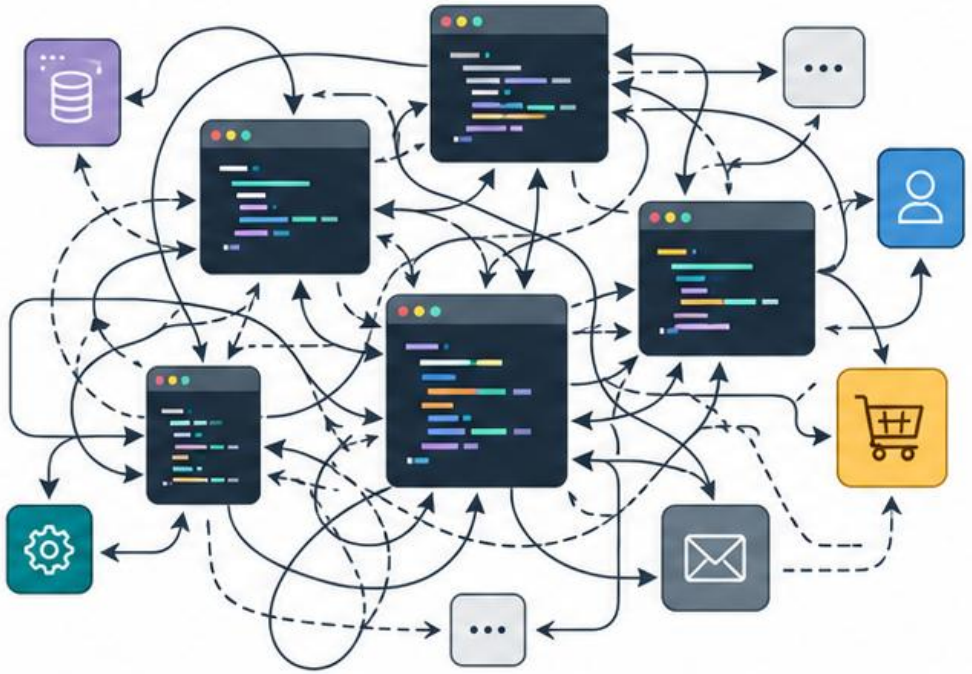


Why Are We Learning Design Patterns?

Design patterns help manage growing software complexity.

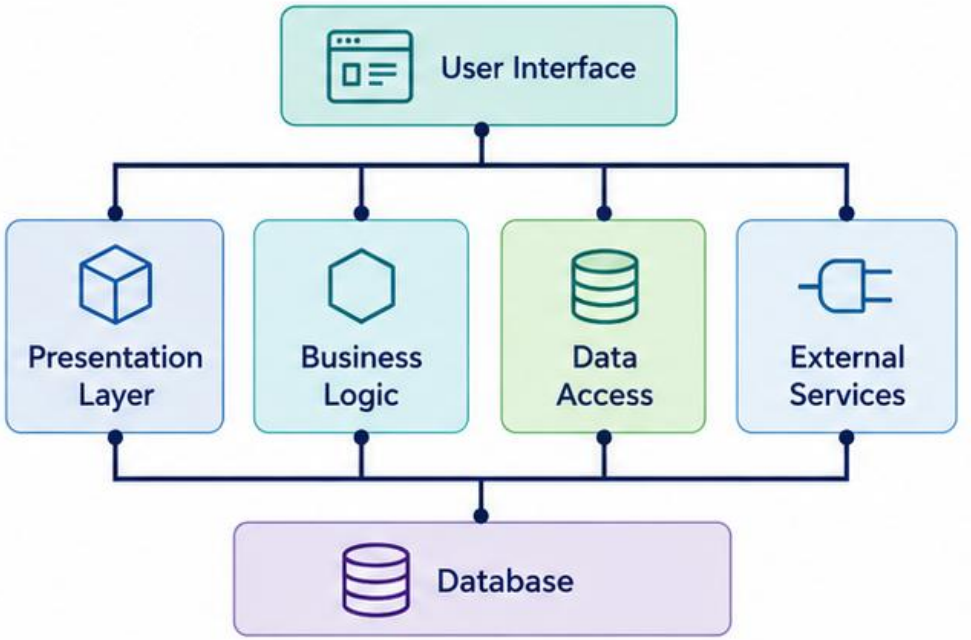
GROWING COMPLEXITY

Hard to understand, change, and maintain



DESIGN PATTERNS BRING ORDER

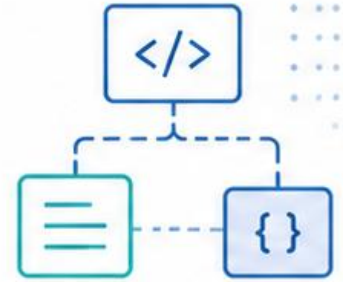
Structured, modular, and easier to evolve



- Solve recurring problems
- Improve readability & maintainability
- Reduce duplication
- Communicate design ideas
- Build flexible & reusable solutions

Learning Outcomes

By the end of this session, students should be able to:





1. Explain what a design pattern is



2. Identify why design patterns are useful in software development



3. Describe the origin of design patterns



4. Differentiate between patterns and anti-patterns



5. Recognize the three main categories of GoF design patterns



6. Understand how the flipped classroom approach will be used in this module

How This Flipped Classroom Will Work

This module will follow a flipped classroom approach.

1 Before Class



Students study given materials such as slides, videos, readings, or code examples.

2 During Class



We focus on discussions, problem-solving, code analysis, quizzes, and activities.

3 After Class



Students reflect, revise, improve solutions, and apply the concepts in practical scenarios.

Key Message

Class time is not mainly for listening.
It is for applying, questioning, and improving understanding.

Student Responsibility in a Flipped Class

To benefit from this module, students are expected to:



1. Study pre-class materials before coming to class



2. Attempt pre-class quizzes or preparation tasks



3. Actively participate in class discussions



4. Work with peers during activities



5. Ask questions when concepts are unclear



6. Apply patterns in coding and design tasks



REMEMBER

Design patterns cannot be learned only by memorizing definitions. They must be practiced.

Assessment Overview

How students will be evaluated in this module

Assessment Method	Description	Allocation
1 Pre-Class / In-Class Quiz	• Based on preparation and early understanding	20%
2 Post-Class / In-Class Activities	• Based on application, discussion, and problem solving	30%
3 Final Exam	• Open-book examination	50%

Key Message

Continuous participation is important because design patterns are best learned through repeated practice.



What Is a Design Pattern?

1. A common problem

2. The context in which the problem occurs

A practical, proven, and reusable solution to a recurring software design problem.

3. A general solution structure

4. The consequences of using that solution

Simple Definition: A design pattern is a reusable design idea for solving a common software problem.

Patterns Change with Time & Environment

Cricket strategies and playing styles evolve across eras, formats, and conditions.

CLASSIC ERA

Test Cricket – Traditional Approach



Patience. Technique. Consistency.
Building innings, valuing partnerships, and wearing down the opposition.

ONE-DAY ERA

ODIs – Balance of Risk & Control



Balance. Timing. Adaptability.
Scoring opportunities, rotating strike, and accelerating in phases.

T20 ERA

T20s – Aggression & Impact



Power. Intent. Impact.
High scoring, boundary hitting, and match-winning moments.

Different times, different conditions, same game — evolving patterns of play.

Patterns Exist Everywhere

Recurring solutions appear across many fields.



Construction



Chess



Films



Medicine



Meals

Warm-Up Question – How Do You Cook?



Raw Ingredient



Cooked Dish

- Follow a recipe
- Adjust ingredients
- Family methods
- Time, tools & environment

Can different methods still lead to a correct result?

From Cooking Recipes to Software Patterns

COOKING (RECIPES)



Recipe = proven way to solve a common cooking problem

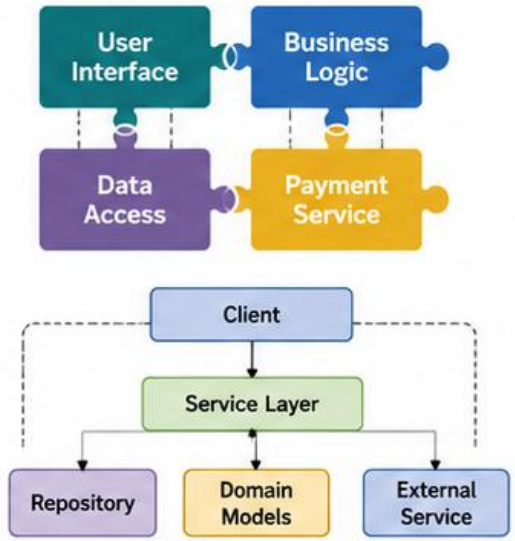
ANALOGY (SAME IDEA)

SOFTWARE (DESIGN PATTERNS)

```
class OrderService {
  privete PaymentProcessor pp;

  public void placeOrder() {
    pp.process(payment);
  }
}
```

```
interface PaymentProcessor {
  void process(Payment p);
}
class CreditCardProcessor
  implements PaymentProcessor {
  public void process(Payment p) {
    // process credit card
  }
}
```



Design pattern = proven way to solve a common software design problem

A design pattern is not a fixed answer.
It is a reusable guide that helps us make better design decisions.

BOTH RECIPES AND DESIGN PATTERNS MAY NEED ADJUSTMENT

INGREDIENTS
Vary by availability and quality

TOOLS
Different tools can change the outcome

REQUIREMENTS
Goals and constraints may be different

CONTEXT
Environment and situation influence the choice

Understand the problem. Consider the context. Choose and adapt the pattern (or recipe) that fits best.

Patterns Change with Environment and Time

Solutions that worked in the past may not always be suitable today.

OLDER APPROACH (PAST)

Simple, Closed, On-Premise



Desktop Computer



On-Premise Server

Monolithic Application

Closed System
(Limited Access & Integration)

EVOLUTION

From monolithic, closed systems to flexible, scalable, and intelligent ecosystems.

MODERN APPROACH (TODAY)

Dynamic, Distributed, Context-Driven



Cloud Infrastructure



Mobile Applications



Web Platforms



Microservices



Modern Architecture
Context-Driven, Scalable, Resilient



APIs and Integrations



AI-based Systems



Security and Scalability Needs

Software design has changed due to:

1. Cloud computing
2. Mobile applications
3. Web platforms
4. Microservices
5. APIs and integrations
6. AI-based systems
7. Security and scalability needs

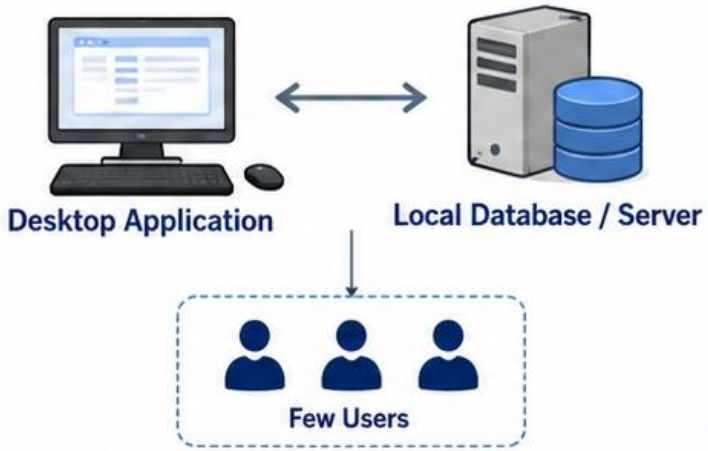


Key Message: Good software design depends on context.

How Software Design Has Changed

Software systems have evolved from small, isolated applications to large, connected, fast-changing ecosystems.

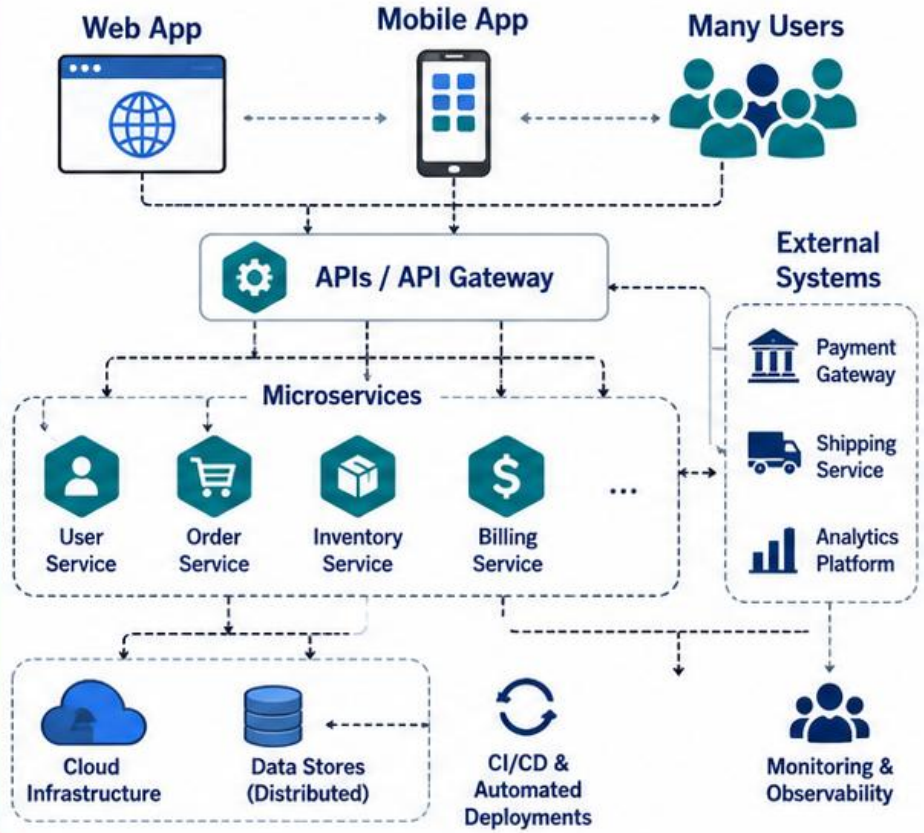
Earlier Software Systems



- Smaller scale
- Isolated
- Easier to manage

Evolution

Modern Software Systems



- Distributed across many services
- Connected to external systems
- Used by many types of users
- Expected to scale quickly
- Frequently updated
- Maintained by teams over time

Problem: **Poor design decisions can create long-term maintenance difficulties.**

What Design Patterns Are Not

Design patterns are not:

1. 
   Ready-made code that can be copied everywhere

2. 
   A replacement for programming knowledge

3. 
   A guaranteed perfect solution

4. 

   A rule that must always be followed

5. 
   A way to make simple problems unnecessarily complex



Key Message: A design pattern should be used only when it improves the design.

Why Basic OOP Concepts Are Not Always Enough

OOP provides powerful building blocks, but good software design requires more than knowing the basics.

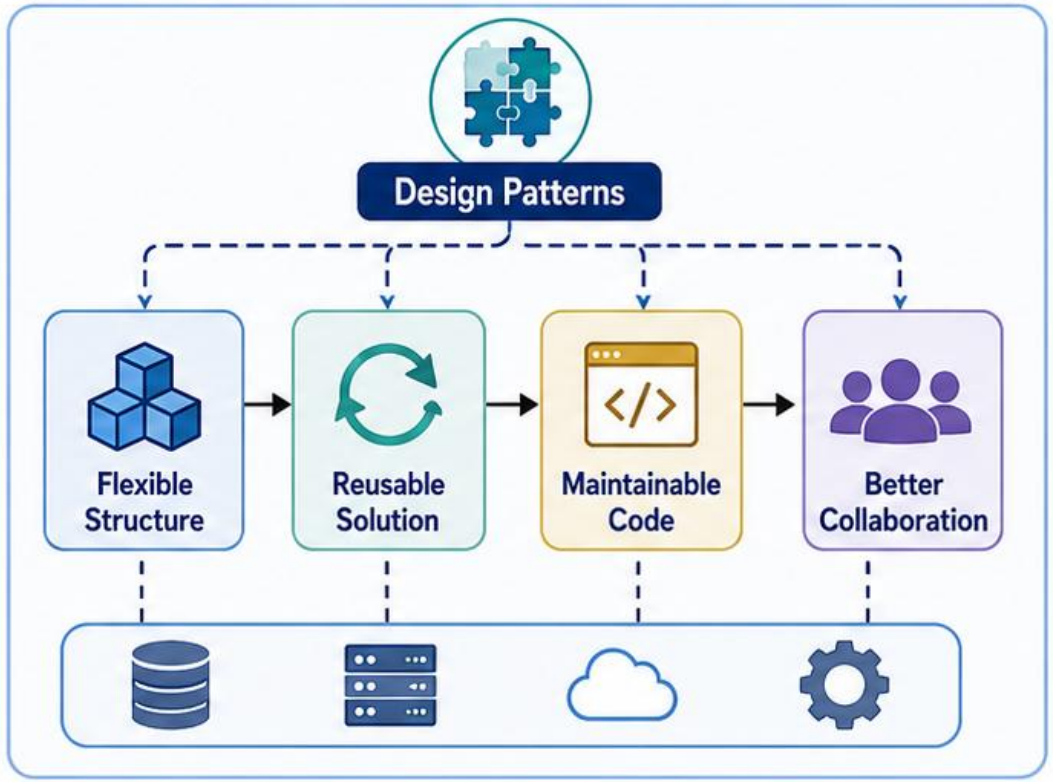
Basic OOP Tools

- Classes
- Objects
- Inheritance
- Interfaces
- Encapsulation
- Polymorphism

Knowing the tools is not enough

Programming constructs alone do not guarantee good design.

Design Patterns in Real Design Situations



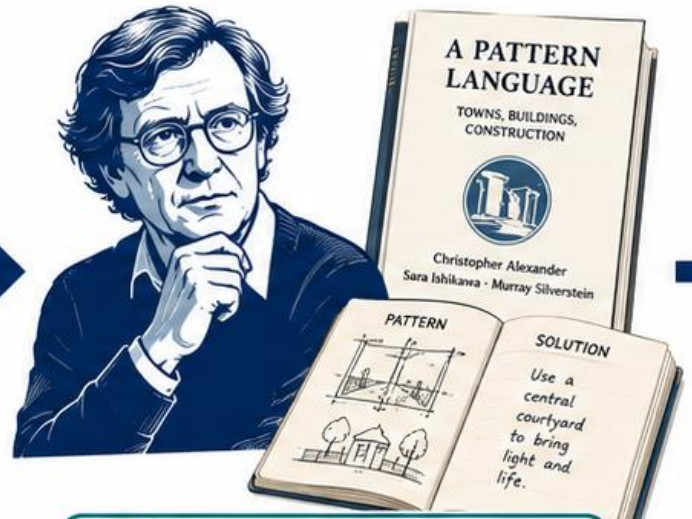
Design patterns show us how to use OOP concepts effectively in real design situations.

Origin of Design Patterns

1 Building Architecture



2 Christopher Alexander

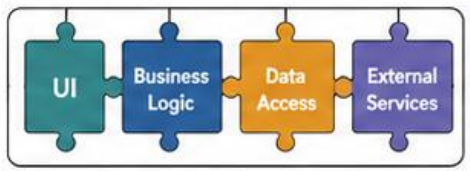


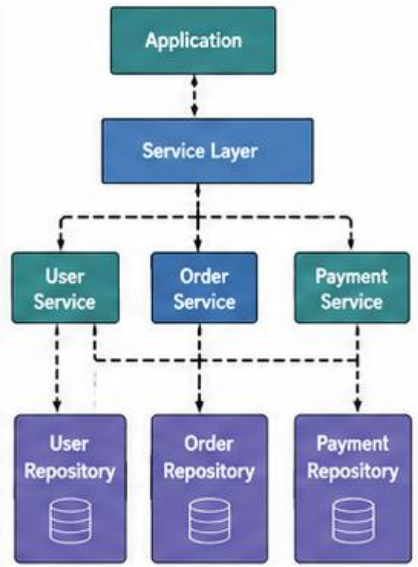
Reusable patterns for recurring design problems

3 Software Engineering

```
class OrderService {
  private PaymentProcessor payment;
  private InventoryService inventory;

  public Order place(Order order) {
    inventory.reserve(order);
    payment.charge(order);
    return order;
  }
}
```





The idea of design patterns first became popular in **building architecture**.

Christopher Alexander introduced the idea that common design problems in buildings and towns can be solved using **reusable patterns**.

Later, this idea was adapted to **software engineering**.

Key Message: Design patterns began as a way of capturing proven design knowledge.

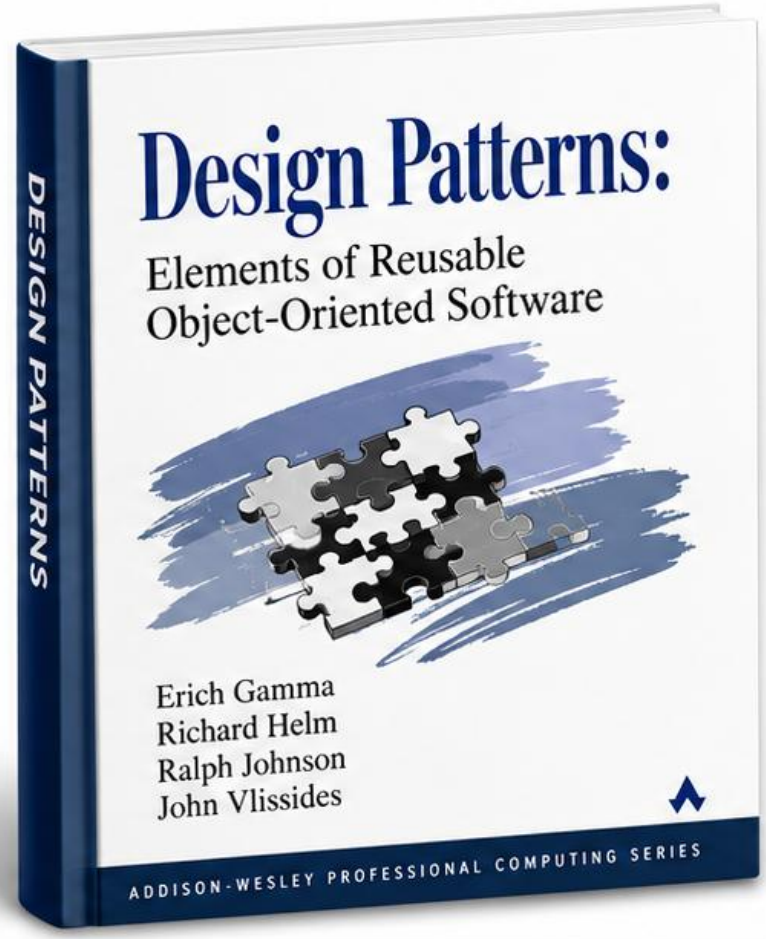
Gang of Four Design Patterns

1995

In 1995, four authors published the influential book.

They are commonly known as the **"Gang of Four".**

The book introduced **23** well-known object-oriented design patterns.



The Four Authors – The "Gang of Four"


Erich Gamma


Richard Helm


Ralph Johnson


John Vlissides

23 Design Patterns

These patterns are now known as GoF Design Patterns.

UNIVERSITY

Why GoF Patterns Are Important

GoF patterns help developers design software more clearly and effectively.

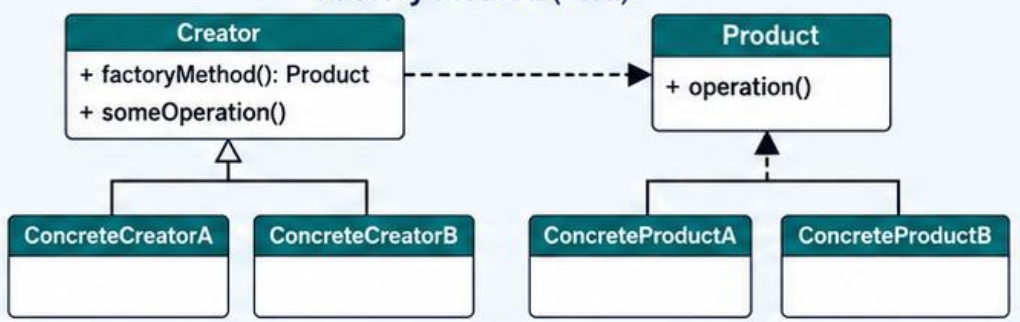
GoF Patterns

1. **Provide a common vocabulary for developers**
2. **Help teams discuss design decisions clearly**
3. **Encourage reusable and flexible design**
4. **Reduce repeated design mistakes**
5. **Support better software architecture**

Example:

Instead of explaining a long object-creation process, a developer can say, 'We can use the **Factory Method pattern.**'

Factory Method (Idea)





Creates objects without specifying the exact class.

Patterns help teams communicate faster and design better.

UNIVERSITY

Design Patterns Need Judgment

Design patterns do not guarantee an absolute solution.



Use judgment when applying patterns

1 Modify the pattern

Adapt it to the problem.

2 Combine patterns

Use more than one when needed.

3 Avoid unnecessary complexity

Do not force a pattern.

4 Consider context, team skill, and future maintenance

Design for the real situation.

Key Message: Design patterns are tools for thinking, not rules for memorizing.

What Are Anti-Patterns?

An anti-pattern is a common but poor solution to a recurring problem.

Short term

Seems useful now

Long term

Creates future problems

Why Do Anti-Patterns Happen?

- Time pressure
- Poor planning
- Overconfidence
- Misuse of technology
- Lack of design thinking
- Ignoring future maintenance

Anti-patterns may offer quick fixes, but they often lead to long-term design and maintenance issues.

Examples of Anti-Pattern Thinking

Common statements that may lead to anti-patterns:



“We need to deliver quickly; design is not important now.”



“We do not need to analyze the impact.”



“I know design patterns, so I can reuse anything.”



“Let us use the latest technology just to impress users.”





“Complicated code proves that I am skilled.”



Discussion Question: Have you seen code that worked but was very difficult to understand or change?



Anti-pattern thinking often favors short-term convenience over long-term quality.

Categories of GoF Design Patterns

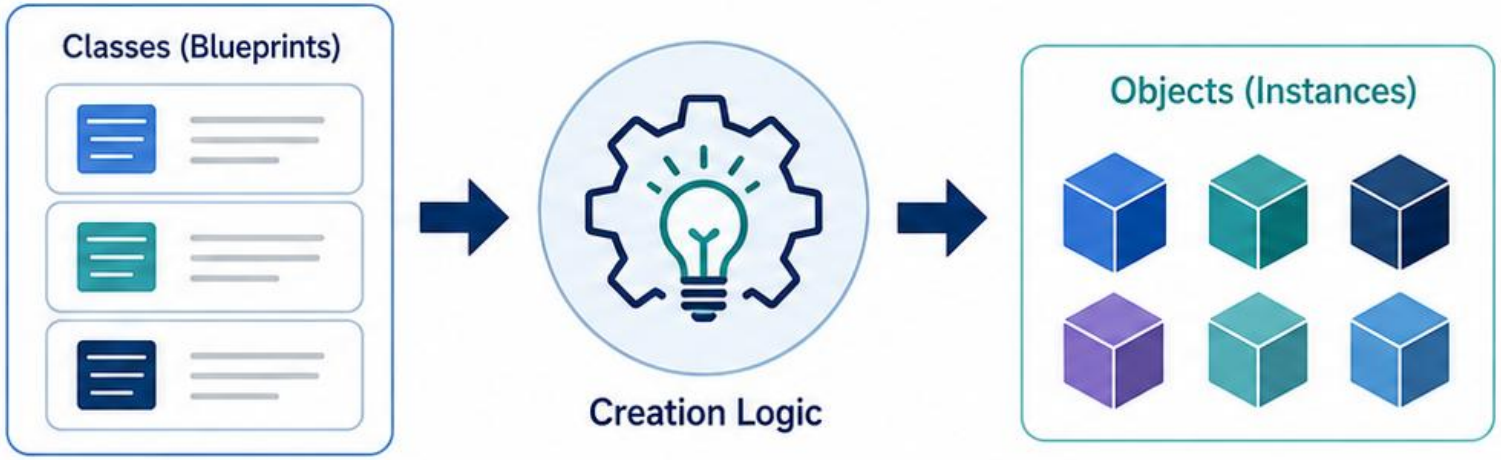
GoF patterns are grouped into three main categories.

Pattern Type	Main Purpose	Simple Meaning
1. Creational Patterns	Object creation	How objects are created
2. Structural Patterns	Object composition	How classes and objects are connected
3. Behavioral Patterns	Object interaction	How objects communicate and behave

These three categories help us understand how design patterns solve different kinds of software design problems.

Creational Design Patterns

Creational patterns focus on object creation.



They help us decide:

1. When to create objects
2. How to create objects
3. Who should create them
4. How to reduce dependency on specific classes

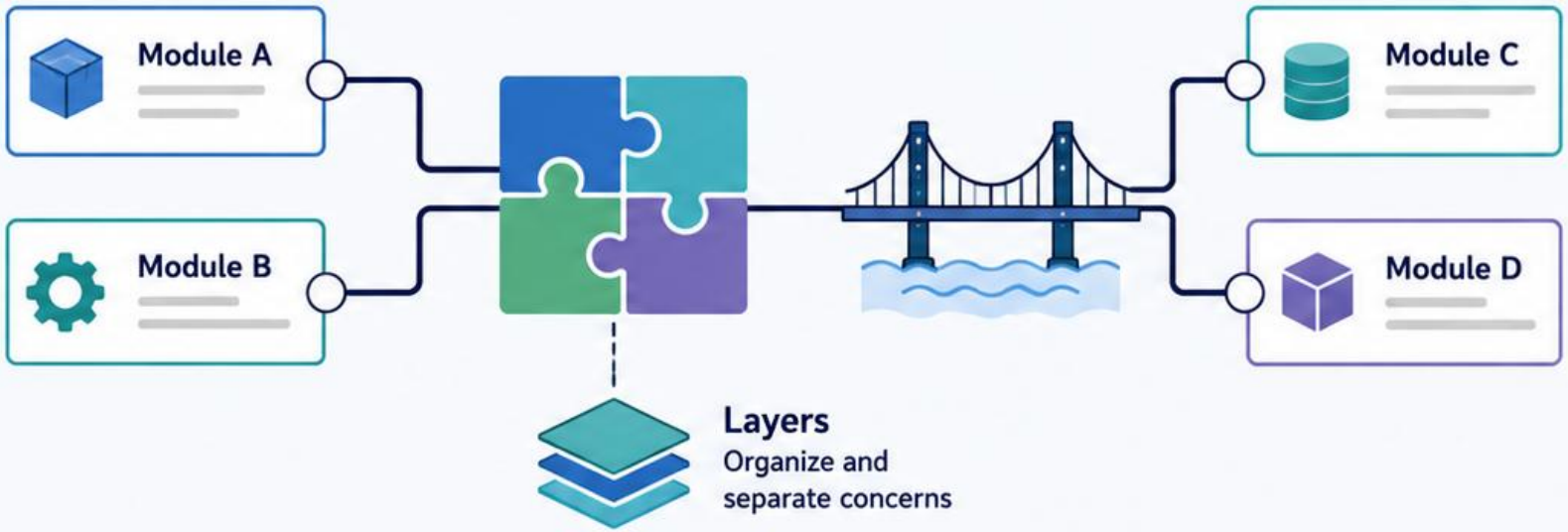
Examples:

- Singleton
- Factory Method
- Abstract Factory
- Builder
- Prototype

Simple Question: How should objects be created in a flexible way?

Structural Design Patterns

Structural patterns focus on how classes and objects are combined.



They help us:

- Connect different parts of a system
- Reuse existing code
- Hide system complexity
- Build flexible software structures

Examples:

- Adapter
- Decorator
- Facade
- Composite
- Proxy
- Bridge
- Flyweight

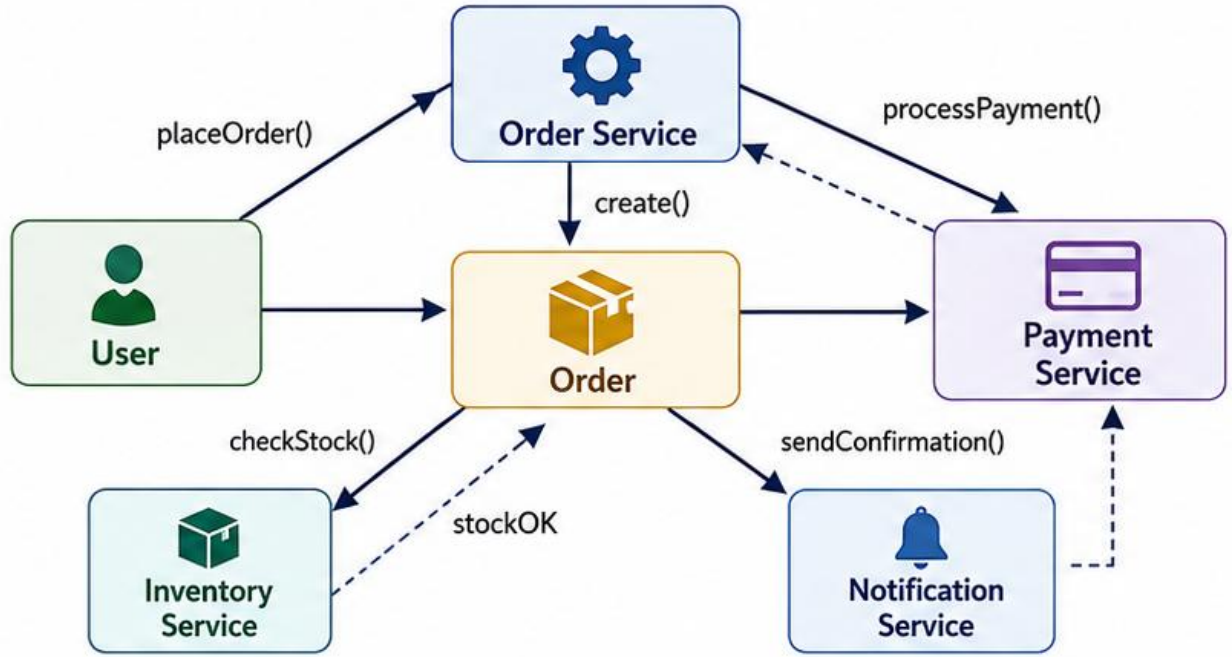
Simple Question: How should classes and objects be connected in a flexible way?

Behavioral Design Patterns

Behavioral patterns focus on communication between objects.

They help us:

1. Assign responsibilities clearly
2. Manage object interaction
3. Reduce tight coupling
4. Make behavior easier to change



- Examples:

Observer | Strategy | Command | Iterator | State | Template Method | Chain of Responsibility

Simple Question: *How should objects work together to complete a task?*

Simple Example Scenario

Online Food Ordering System

CUSTOMER



Customer places an order via mobile app or web app.

ORDER SYSTEM

Receives order, validates, manages order details and status, and coordinates with services.

PAYMENT SERVICE
Process payments securely.

DELIVERY SERVICE
Assign rider and track delivery.

NOTIFICATION SERVICE
Send order updates via push, SMS, or email.

DISCOUNT STRATEGY / PRICING
Apply pricing rules and discounts.

2 STRUCTURAL

Connect payment, delivery, and notification services.

3 BEHAVIORAL

Notify customers when order status changes and allow different discount strategies.

1 CREATIONAL

Create different types of orders.

By Fulfillment Type	By Order Scheduling
Dine-in	Standard Order
Takeaway	Scheduled Order
Delivery	Group Order

ORDER STATUS FLOW

Order Placed → Confirmed → Preparing → Out for Delivery → Delivered / Completed

Discussion: Which parts of this system involve object creation, structure, and behavior?

In-Class Activity: Design Cases

Read each case and identify the design problem.

For each case, discuss:

1) What is the recurring design problem?
2) Which part may become difficult to maintain?
3) Is it about object creation, structure, or behavior?
4) What type of design pattern may help?

1



Multiple Payment Methods

An online store needs card, bank transfer, mobile wallet, and cash-on-delivery options.

2



Different User Accounts

A university system must create student, lecturer, admin, and guest accounts.

3



Order Update Notifications

A food delivery app must notify customers when order status changes.

4



New Payment Gateway

An e-commerce system must connect to a third-party payment gateway with a different interface.

5



Complex Student Report

A student management system builds reports with many optional sections.

6



Different Discount Rules

An online shopping system supports seasonal, loyalty, and coupon discounts.

7



Library System Simplification

A mobile app needs simple access to many complex library management modules.

8



Shared Configuration Manager

A software system should use only one shared configuration manager.

9



Undo and Redo

A text editor must support undo and redo for user actions.

10



Changing Order Status Behavior

An online order behaves differently when pending, shipped, or delivered.

Discuss in groups and present your reasoning.

IN-CLASS ACTIVITY: DESIGN CASES WITH PATTERN TYPE

#	DESIGN CASE (SCENARIO)	PATTERN TYPE (CATEGORY)	EXAMPLE PATTERN(S)
1	Multiple Payment Methods in an Online Store	Behavioral / Creational	Strategy, Factory Method
2	Creating Different Types of User Accounts	Creational	Factory Method, Abstract Factory
3	Notification System for Order Updates	Behavioral	Observer
4	Connecting a New Payment Gateway	Structural	Adapter
5	Building a Complex Student Report	Creational	Builder
6	Applying Different Discount Rules	Behavioral	Strategy
7	Simplifying Access to a Complex Library System	Structural	Facade
8	Restricting a System to One Configuration Manager	Creational	Singleton
9	Undo and Redo in a Text Editor	Behavioral	Command
10	Changing Order Status Behavior	Behavioral	State
11	Adding Extra Features to a Basic Coffee Order	Structural	Decorator
12	Processing Student Requests Through Several Officers	Behavioral	Chain of Responsibility

Design Pattern Mindset

When studying a design pattern, do not only memorize the name.

1. What problem does it solve?
2. When should it be used?
3. When should it not be used?
4. What are the advantages?
5. What are the disadvantages?
6. How does it improve the design?
7. What happens if we do not use it?



Think beyond the pattern name — understand the problem, context, trade-offs, and impact.

Key Takeaways

Today we learned that:

1.  Design patterns are reusable solutions to recurring software design problems

2.  They support better communication, flexibility, and maintainability

3.  They are not fixed code templates

4.  They must be selected based on context

5.  Anti-patterns are poor solutions that create long-term problems

6.  GoF patterns are grouped as creational, structural, and behavioral patterns



Good design is not about memorizing patterns — it is about choosing the right solution for the right context.

Closing Thought

> Good software is not only about making the program work.



Understandable



Changeable



Reusable



Testable

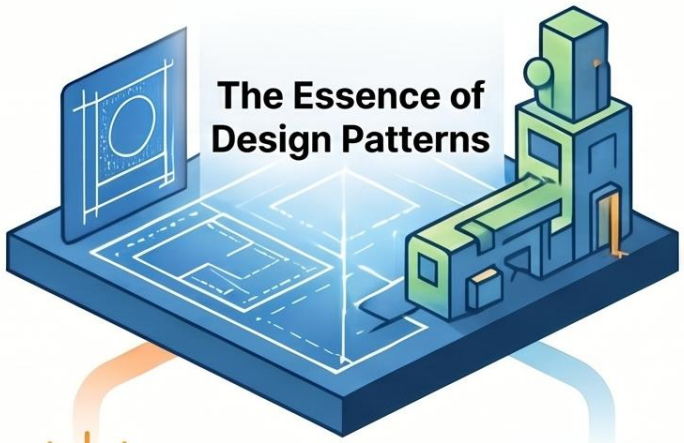


Maintainable



Design patterns help us design software that can survive change.

Mastering Software Design Patterns



Reusable Solutions for Common Problems

A pattern is a practical, proven guide for solving recurring software design challenges.

Managing Growing System Complexity

Patterns bring order to codebases that are otherwise hard to understand or maintain.



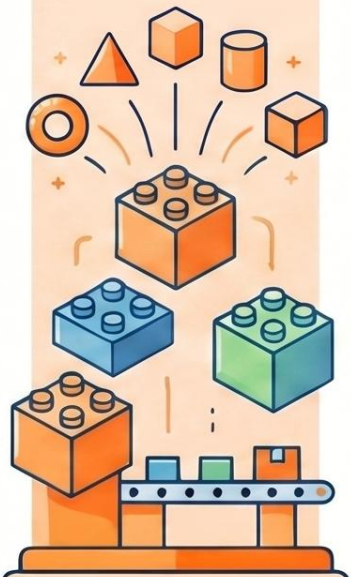
Tools for Thinking, Not Rigid Rules

Patterns require judgment and should be adapted to the specific context of the problem.

The Three Pillars of GoF Patterns

Creational: Flexible Object Creation

Focuses on the logic of how objects are created and who creates them.

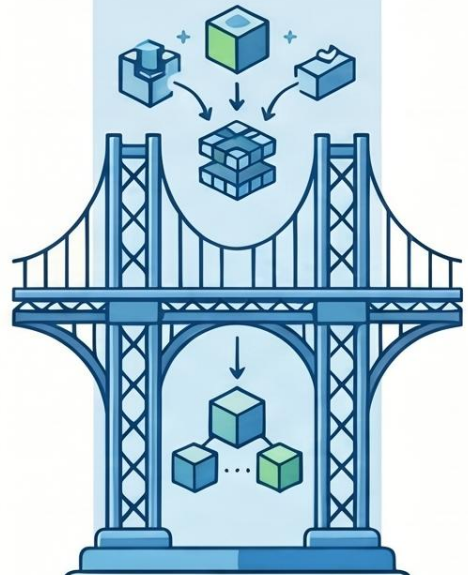


Purpose:
Object creation

Simple Meaning:
How objects are created

Structural: Organizing System Components

Focuses on how classes and objects are composed and connected to form larger structures.



Purpose:
Object composition

Simple Meaning:
How classes and objects are connected

Behavioral: Managing Object Interaction

Focuses on how objects communicate, behave, and assign responsibilities to complete tasks.



Purpose:
Object interaction

Simple Meaning:
How objects communicate and behave

NotebookLM